```
# ====================================
# IMPORTS (FORECASTING)
# ====================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv1D, LSTM, Dense, Dropout,
BatchNormalization
from tensorflow.keras.regularizers import l2
from tensorflow.keras.callbacks import EarlyStopping

# ====================================
# LOAD DATA
# ====================================
df = pd.read_csv("microgrid_data.csv")

print(df.head())
print(df.columns)

df['Time'] = pd.to_datetime(df['Time'])

df['hour'] = df['Time'].dt.hour
df['day'] = df['Time'].dt.day
df['month'] = df['Time'].dt.month
```

```
df['Season'] = df['Season'].astype('category').cat.codes
df['Day_of_the_week'] = df['Day_of_the_week'].astype('category').cat.codes

df['load_lag_1'] = df['Electric_demand'].shift(1)
df['load_lag_2'] = df['Electric_demand'].shift(2)
df['load_rolling_mean'] = df['Electric_demand'].rolling(3).mean()

df = df.dropna()

features = [
    'DHI',
    'DNI',
    'GHI',
    'Wind_speed',
    'Humidity',
    'Temperature',
    'Season',
    'Day_of_the_week',
    'hour',
    'day',
    'month',
    'load_lag_1',
    'load_lag_2',
    'load_rolling_mean'
]

targets = [
```

```
    'PV_production',
    'Wind_production',
    'Electric_demand'
]

X = df[features].values
y = df[targets].values

scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()

X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y)

def create_sequences(X, y, time_steps=12):
    Xs, ys = [], []
    for i in range(len(X) - time_steps):
        Xs.append(X[i:i+time_steps])
        ys.append(y[i+time_steps])
    return np.array(Xs), np.array(ys)

X_seq, y_seq = create_sequences(X_scaled, y_scaled, time_steps=12)

split = int(0.8 * len(X_seq))

X_train, X_test = X_seq[:split], X_seq[split:]
y_train, y_test = y_seq[:split], y_seq[split:]
```

```
input_layer = Input(shape=(X_train.shape[1], X_train.shape[2]))

cnn = Conv1D(
    filters=32,
    kernel_size=3,
    activation='relu',
    kernel_regularizer=l2(0.001)
)(input_layer)

cnn = BatchNormalization()(cnn)
cnn = Dropout(0.3)(cnn)

lstm = LSTM(
    32,
    return_sequences=False,
    kernel_regularizer=l2(0.001)
)(cnn)

lstm = Dropout(0.3)(lstm)

dense = Dense(16, activation='relu')(lstm)
dense = Dropout(0.2)(dense)

output = Dense(y_train.shape[1])(dense)

model = Model(inputs=input_layer, outputs=output)

model.compile(optimizer='adam', loss='mse')
```

```
model.summary()

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_data=(X_test, y_test),
    callbacks=[early_stop]
)

plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Loss Curve")
plt.show()

y_pred = model.predict(X_test)

y_pred = scaler_y.inverse_transform(y_pred)
y_test_actual = scaler_y.inverse_transform(y_test)
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Split targets
y_true_pv = y_test_actual[:, 0]
y_true_wind = y_test_actual[:, 1]
y_true_load = y_test_actual[:, 2]

y_pred_pv = y_pred[:, 0]
y_pred_wind = y_pred[:, 1]
y_pred_load = y_pred[:, 2]

# Metrics
def evaluate(true, pred, name):
    mae = mean_absolute_error(true, pred)
    rmse = np.sqrt(mean_squared_error(true, pred))
    r2 = r2_score(true, pred)

    print(f"\n 🔹 {name}")
    print("MAE:", mae)
    print("RMSE:", rmse)
    print("R2:", r2)

evaluate(y_true_pv, y_pred_pv, "PV Production")
evaluate(y_true_wind, y_pred_wind, "Wind Production")
evaluate(y_true_load, y_pred_load, "Electric Demand")

plt.plot(y_true_load[:100], label='Actual Load')
plt.plot(y_pred_load[:100], label='Predicted Load')
```

```
plt.legend()
plt.title("Load Forecasting")
plt.show()

model.save("final_microgrid_model.h5")

# Train R2
y_train_pred = model.predict(X_train)
y_train_pred = scaler_y.inverse_transform(y_train_pred)
y_train_actual = scaler_y.inverse_transform(y_train)

print("Train R2:", r2_score(y_train_actual, y_train_pred))
print("Test R2:", r2_score(y_test_actual, y_pred))

plt.scatter(y_true_load, y_pred_load)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Load Prediction")
plt.show()

# ========================================
# IMPORTS
# ========================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
```

```
import gym
from gym import spaces
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv

# ========================================
# LOAD DATA (Database.csv)
# ========================================
df = pd.read_csv("Database.csv", on_bad_lines='skip', engine='python')
df.columns = df.columns.str.strip()

df = df[['PV_production','Wind_production','Electric_demand']].dropna()

data = df.values

# Normalize for RL stability
scaler = MinMaxScaler()
data_norm = scaler.fit_transform(data)

# ========================================
# MICROGRID ENVIRONMENT (NO BATTERY)
# ========================================
class MicrogridEnv(gym.Env):
    def __init__(self, data):
        super().__init__()

        self.data = data
```

```
    self.max_steps = len(data)
    self.step_id = 0

    # state: solar, wind, load
    self.observation_space = spaces.Box(0,1,(3,),dtype=np.float32)

    # action: weights for solar, wind, grid
    self.action_space = spaces.Box(0,1,(3,),dtype=np.float32)

def reset(self):
    self.step_id = 0
    return self.data[self.step_id]

def step(self, action):

    solar_avail, wind_avail, load = self.data[self.step_id]

    # ----------------------------
    # ACTION → NORMALIZED WEIGHTS
    # ----------------------------
    action = np.clip(action, 0, 1)
    weights = action / (np.sum(action) + 1e-8)

    w_solar, w_wind, w_grid = weights

    demand = load

    # ----------------------------
```

```
# ENERGY DISTRIBUTION (REALISTIC)
# ----------------------------
# Solar first
solar_used = min(solar_avail, w_solar * demand)

# Wind next
remaining_after_solar = demand - solar_used
wind_used = min(wind_avail, w_wind * remaining_after_solar)

# Grid fills remaining demand
remaining = demand - (solar_used + wind_used)
grid_used = max(0, remaining)

total_supply = solar_used + wind_used + grid_used

# ----------------------------
# REWARD FUNCTION (BALANCED)
# ----------------------------
scale = demand + 1e-6

mismatch = abs(total_supply - demand) / scale
renewable = solar_used + wind_used

renewable_ratio = renewable / scale
grid_ratio = grid_used / scale

reward = 0
```

```
# 1. Strong balance
reward += 12 * (1 - mismatch)

# 2. Renewable usage
reward += 6 * renewable_ratio

# 3. Grid penalty (moderate)
reward -= 2.0 * grid_ratio

# 4. Use available solar/wind
if solar_avail > 0.2:
    reward += 2 * (solar_used / (solar_avail + 1e-6))

if wind_avail > 0.2:
    reward += 2 * (wind_used / (wind_avail + 1e-6))

# 5. Penalize ignoring renewables
if solar_avail > 0.2 and solar_used < 0.05:
    reward -= 3

if wind_avail > 0.2 and wind_used < 0.05:
    reward -= 3

# 6. Penalize not using grid when needed
if remaining > 0.05 and grid_used < 0.05:
    reward -= 3

# 7. Bonus for perfect match
```

```
        if mismatch < 0.01:
            reward += 6

        # ----------------------------
        self.step_id += 1
        done = self.step_id >= self.max_steps - 1

        return self.data[self.step_id], reward, done, {}

# ========================================
# CREATE ENV
# ========================================
env = DummyVecEnv([lambda: MicrogridEnv(data_norm)])

# ========================================
# TRAIN PPO
# ========================================
model = PPO(
    "MlpPolicy",
    env,
    learning_rate=2e-4,
    n_steps=4096,
    batch_size=128,
    gamma=0.99,
    ent_coef=0.03,
    verbose=1
)
```

```
model.learn(total_timesteps=80000)

# ========================================
# TEST MODEL
# ========================================
test_env = MicrogridEnv(data_norm)
obs = test_env.reset()

print("\n ⚡ OPTIMIZED ENERGY DISTRIBUTION\n")

solar_log, wind_log, grid_log, reward_log = [], [], [], []

for i in range(20):
    action, _ = model.predict(obs)

    obs, reward, done, _ = test_env.step(action)

    solar_avail, wind_avail, load = data_norm[i]

    weights = action / (np.sum(action) + 1e-8)
    w_s, w_w, w_g = weights

    solar_used = min(solar_avail, w_s * load)
    wind_used = min(wind_avail, w_w * (load - solar_used))
    grid_used = max(0, load - (solar_used + wind_used))

    solar_log.append(solar_used)
    wind_log.append(wind_used)
```

```
    grid_log.append(grid_used)
    reward_log.append(reward)

    print(f"Step {i+1}")
    print(f"Demand: {load:.3f}")
    print(f"Solar Used: {solar_used:.3f}")
    print(f"Wind Used : {wind_used:.3f}")
    print(f"Grid Used : {grid_used:.3f}")
    print(f"Renewable %: {(solar_used+wind_used)/load*100:.2f}%")
    print(f"Reward: {reward:.3f}")
    print("-"*40)

    if done:
        break

# ========================================
# GRAPHS
# ========================================
plt.figure(figsize=(10,5))
plt.plot(solar_log,label="Solar")
plt.plot(wind_log,label="Wind")
plt.plot(grid_log,label="Grid")
plt.legend()
plt.title("Balanced Energy Distribution")
plt.grid()
plt.show()

plt.figure(figsize=(10,4))
```

```
plt.plot(reward_log)
plt.title("Reward Curve")
plt.grid()
plt.show()
```